



By Yahoo Baba

Mastering React.js: A Comprehensive Guide to Modern Web Development

1 Create Project & Folder Structure

2 Component

3 Component with props

4 Destructuring props

5 JSX

6 Component Conditional Rendering

7 Conditional Rendering Object & Array

8 Events Handling

9 useState

10 From input

11 From Multiple input

12 From Submit

13 From textarea

14 From select

15 CSS Inline Style

16 CSS Stylesheet

20 Hook useRef DOM Manipulation

17 CSS Module

21 Hook useRef none UI impacting data

18 Hook useEffect

22 Hook useReducer

19 Hook useContext

23 Hook useMemo

24 Hook useCallback

25 Hook Custom hook : useFetchData

React Create Project & Folder Structure

1

1 `npm create vite@latest` (Create React App)

> node_modules

> public

✓ src

> assets

App.css

App.jsx

index.css

main.jsx

.gitignore

eslint.config.js

index.html

package-lock.json

package.json

README.md

vite.config.js

A folder used to store static assets like images, fonts that don't need processing by Vite. Any file in this folder is copied directly to the final build without any modification.

The `src` folder contains all the source code of the project.

Stores project assets like images, SVGs, etc.

A CSS file for styling the main `App.jsx` component.

The main component of the React application.

Global styles that apply to the entire app.

The entry point of the React application, usually where React is initialized and linked to the root HTML element (often found in `index.html`).

The main HTML file that serves the React application. It contains the root element (`<div id="root">`) where React will inject the components.

Contains metadata about the project and its dependencies. It includes scripts, versions, and details of the project's dependencies.

2 `App.jsx`

```
function App() {  
  return (  
    <h1> Hello React Learner</h1>  
  );  
}  
  
export default App;  
//entry point of react file
```

3 `Main.jsx`

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import App from './App';  
  
const root =  
  ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <React.StrictMode> <App /> </React.StrictMode>  
>); //render App.jsx index.html file root div
```

4 `npm run dev`

localhost:5173/

Hello React Learner

1 Create folder name : **component** & in this folder create a file name : **Message.jsx**

2 component\Message.jsx

```
function Message() { //functional component, component name always first letter in upper case
  return (
    <h1> Hello React Learner Component</h1> //write jsx code inside of return block
  );
}

export default Message; //export your component
```

3 App.jsx

```
import Message from './component/Message'; //import Message component first

function App() {

  return ( <Message/> ); //call your component
}

export default App;
```

A **React component** is a reusable, independent piece of code in React that represents a part of the user interface (UI). Components in React allow you to break down the UI into smaller, manageable pieces that can be developed, maintained, and reused individually.

Functional Components:

Written as JavaScript functions.

They accept properties (props) as arguments and return JSX (a syntax that looks like HTML).

localhost:5173/

Hello React Learner

React Component with props

3

1 Create folder name : **component** & in this folder create a file name : **Message.jsx**

2 component\Message.jsx

```
function Message(props) { //receive data when component call by props
  return (
    <h1> {props.msg}</h1> //getting msg attribute data using props
  );
}

export default Message; //export component
```

3 App.jsx

```
import Message from './component/Message'; //import Message component first

function App() {
  return ( <Message msg="Hello React Learner"/> ); //send data to component using attribute
}
export default App;
```

Props (short for properties) in React are a way of passing data from one component to another. They are read-only and are used to give components dynamic data.

In the example we provided, props are passed from the App component to the Message component:

localhost:5173 /

Hello React Learner

React Destructuring props

4

1 Create folder name : **component** & in this folder create a file name : **Hello.jsx**

2 App.jsx

```
import Hello from "../component/Hello"; //import Hello component first

function App() {
  return (
    <>
      <Hello name="Anjesh" message="Hello" /> //send data to component using attributes
      <Hello name="Abhiyansh" message="Hello" /> //send data to component using attributes
    </>
  );
}

export default App;
```

3 component\Hello.jsx

```
function Hello(props) {
  return ( <h1> {props.message} {props.name} </h1> ); //normal way to read props data
}

-----

function Hello({ name, message }) { //read data using destructing method pass attribute name
  //in {name,message}
  return ( <h1> {message} {name} </h1> );
}

-----

function Hello(props) {
  const { name, message } = props; //2nd way to destructor props value
  return ( <h1> {message} {name} </h1> );
}

export default Hello;
```

In React, destructuring props is a technique where you extract specific properties from the props object passed to a component, making it easier to access and use them within the component. Instead of referencing props.propertyName each time you need to use a prop, destructuring allows you to directly access the property by its name.

Definition:

Destructuring props is the process of unpacking properties from the props object directly into distinct variables, either within the function argument or in the function body. It improves code readability and reduces repetition

localhost:5173

Hello Anjesh
Hello Abhiyansh

1 Create folder name : **component** & in this folder create a file name : **Message.jsx**

2 App.jsx

```
import Message from "../component/Message"; //import Message component first
function App() {
  return <Message />; //call your component
}
export default App;
```

3 component\Message.jsx

```
function Message() {
  return (
    <div>
      <p>Hello React Learners</p> //in side return block all look like html but is a jsx
    </div>                      jsx allow to write html and JavaScript together
  );                            you can write any js code in {} in jsx code block
}

-----

const name = "Anjesh Kumar";

function Message() {
  return <p>Hello React Learners {name}</p>; //if you want to write js code in jsx use {}
} //in this example we show name variable value in p tag using {JavaScript code}

-----

const user = {
  name:"Anjesh Kumar",
  age:25,
  city:'Delhi'
};

function Message() {
  return <p>User : {user}</p>; //in this example show object in jsx
}

-----

function Message() {
  return <p>Total React Learners {10 + 10}</p>; //JavaScript express in {}
}

-----

function hello() {
  return "React";
}

function Message() {
  return <p>Hello {hello()} Learners</p>; //call function in jsx
}

-----

function Fruits() {
  const fruits = ["Mango", "Banana", "Watermelon", "Papaya", "Coconuts"];
  return <div> {fruits.map(fruit => <h1>{fruit}</h1>)} </div>
} //using map method to display array value in jsx code block

export default Message;
```

In React, JSX (JavaScript XML) is a syntax extension of JavaScript used to describe the UI structure of components. It looks similar to HTML but is more powerful, as it allows embedding JavaScript expressions directly within the markup. JSX gets transpiled to React.createElement() function calls by tools like Babel, making it understandable to browsers. Here's a breakdown of JSX:

JSX Definition:

JSX stands for JavaScript XML and is used to build React components. It allows you to write HTML-like syntax in JavaScript files. JSX must be wrapped in a single enclosing tag, such as a div, fragment, or any HTML/React component.

Component Conditional Rendering

1 Create folder name : **component** & in this folder create 2 file name : **Admin.jsx** & **User.jsx**

2 component\Admin.jsx

```
export default function Admin() {
  return <p>This is Admin Component</p>; //make component name Admin
}
```

3 component\User.jsx

```
export default function User() {
  return <p>This is User Component</p>; //make component name User
}
```

4 App.jsx

```
import User from "../component/User"; //import user component
import Admin from "../component/Admin"; //import Admin component

function App() {
  const visitor = "admin";
  return visitor === "adimin" ? <Admin /> : <User />;
}

export default App; //using ternary operator to make condition, if visitor === admin then
return Admin component else return User component
-----

let display;
const visitor = "admin";
if (visitor === "adimin") {
  display = <Admin />;
} else {
  display = <User />;
}
return display; //2nd way of conditional rendering using if-else block
}
-----

let visitor = "user"; //3rd way of conditional rendering using && operator

return visitor === "user" && <User />; //render component if both condition match
-----

let visitor = "admin";
switch (visitor) {
  case "admin":
    return <Admin />;
  case "user":
    return <User />; //4th way of conditional rendering using switch case for multiple
condition
}
```

Summary:

if statements: Best for complex logic with multiple conditions.

Ternary operator (?:): Ideal for simple, two-choice conditions.

Logical &&: Great for rendering something only when a condition is true.

switch case: Useful for handling multiple values or complex branching logic.

Inline if-else: Offers flexibility with short-circuit logic for more complex conditional rendering.

Short-circuiting (||): Provides default behavior when a value is false or null.

Conditional Rendering Object & Array

7

1 Create folder name : **component** & in this folder create 2 file name : **Fruit.jsx** & **Fruits.jsx**

2 component\Fruit.jsx

```
export default function Fruit({ name, color, weight, price }) {  
  return (  
    <>  
      {price > 50 ? <p>{name} - {color} - {weight} - rs {price}</p> : "" }  
    </>  
  ); //in this example conditionally render data in price greater then 50 then render date  
  }                                     else render empty string
```

3 component\Fruits.jsx

```
import Fruit from "../Fruit"; //import Fruit Component  
  
export default function Fruits() {  
  const fruits = [  
    { name: "Apple", color: "Red", weight: 180, price: 120 },  
    { name: "Banana", color: "Yellow", weight: 120, price: 40 },  
    { name: "Orange", color: "Orange", weight: 150, price: 60 },  
    { name: "Grapes", color: "Purple", weight: 50, price: 90 },  
    { name: "Mango", color: "Green", weight: 200, price: 150 },  
  ];  
  
  return (  
    <>  
      {fruits.map((fruit) => (  
        <Fruit name={fruit.name} color={fruit.color} weight={fruit.weight} price={fruit.price}/>  
      ))}  
    </> // render fruits object data using map method show fruit data in Fruit component  
  );  
}
```

4 App.jsx

```
import Fruits from "../component/Fruits"; //import Fruits component  
  
function App() {  
  return <Fruits />;  
}  
  
export default App;
```

localhost:5173

Apple - Red - 180 - rs 120
Orange - Orange - 150 - rs 60
Grapes - Purple - 50 - rs 90
Mango - Green - 200 - rs 150

1 Create folder name : **component** & in this folder create a file name : **Message.jsx**

2 component\Message.jsx

```
export default function Message() { //component name msg

  function showMessage() { //make function in this function define alert box
    alert("Button Click");
  }

  return (
    <div>
      <button onClick={showMessage}>Click</button> //call function using onClick event
    </div>
  ); //onClick={functionName} 1st way to call function on js event
    //onClick={()=>function(parameter)} 2nd way to call function with parameter
}
```

3 App.jsx

```
import Message from "../component/Message"; //import Message component

function App() {
  return <Message />; //call Message component or you can say render
}

export default App;
```

In React, event handling refers to the process of managing user interactions such as clicks, key presses, form submissions, and other types of input events within a React component. React provides a way to handle these events using functions known as event handlers. These handlers are passed as props to the JSX elements in the component and are executed when the specified event occurs.

React event handlers are similar to those in vanilla JavaScript, but there are a few key differences:

Naming Convention: React uses camelCase for event handler names (e.g., `onClick`, `onSubmit`) instead of lowercase (`onclick`, `onsubmit`) as in HTML.

Event Object: In React, the event object is a synthetic event, which is a wrapper around the native event object. This ensures that events work consistently across different browsers.

Binding Event Handlers: In class components, event handlers often need to be explicitly bound to the component instance using `bind(this)`, but this is not necessary when using arrow functions or with functional components and hooks.

localhost:5173

Click

localhost:5173 says
Button Click

OK

1 Create folder name : **component** & in this folder create a file name : **Counter.jsx**

2 component\Counter.jsx

```
import { useState } from "react"; //import useState

export default function Counter() {

  const [count, setCount] = useState(0); //define useState, [variable , updater function]

  function handleClick() { // make function
    setCount(count + 1); //when this function call update count variable using updaterfunction
  }

  return (

    <div>
      <h1>Counter value is : {count}</h1>
      <button onClick={handleClick}>Increment</button> //call function on : onClick event
    </div>

  ); //when function button click handelClick function call in, and function code execute
}
```

3 App.jsx

```
import Counter from "../component/Counter"; //import counter component

function App() {
  return <Counter />; //call Counter component you can say render Counter Component
}

export default App;
```

useState is a React Hook that lets functional components store and manage data that can change, called "state." It provides two things:

Current state: The value you want to track (like a number, text, or object).

Function to update the state: A way to change the value, which automatically updates the UI.

You start by giving it an initial value. When you change the state using the provided function, React automatically re-renders the component to show the new value.

localhost:5173

Counter value is : 0

Increment

1 Create folder name : **component** & in this folder create a file name : **Form.jsx**

2 component\Form.jsx

```
import { useState } from "react";

export default function Form() {
  const [name, setName] = useState(""); //define useState to store and update input value

  return (
    <div>
      <form>

        <label> First Name
        <input type="text" value={name} onChange={(e) => setName(e.target.value)}/>
        </label> //in onChange event we update name variable using updater function

        <h1>{name}</h1> //display name variable value

      </form>
    </div>
  ); //in onChange event make arrow function and pass e as event to read input date by
  event.target.value
}
```

3 App.jsx

```
import React from "react";
import Form from "../component/Form"; //import Form Component

function App() {
  return <Form />; //render the form component
}

export default App;
```

- **Controlled Components:** Input values are tied to React state and updated via onChange.
- **Form Submission:** Handle form submission with onSubmit and prevent default behavior using event.preventDefault().
- **Handling Multiple Inputs:** Use a single state object to manage multiple form fields dynamically.
- **Validation:** Validate input either during onChange or onSubmit.
- **Uncontrolled Components:** Use ref to access input values without managing state.

localhost:5173

First Name

ANJESH KUMAR

From Multiple input

11

1 Create folder name : **component** & in this folder create a file name : **Form.jsx**

2 component\Form.jsx

```
import { useState } from "react"; //import useState hook

export default function Form() {
  const [inputs, setInputs] = useState({}); //initialize useState empty object

  function handleChange(event) { //make function to assign input value to useState Object
    const name = event.target.name; //get target input filed name attribute value
    const value = event.target.value; //get target input value attribute value
    setInputs((values) => ({ ...values, [name]: value })); //update input value to useState
  }

  const handleSubmit = (event) => { // checking input value using when form submit
    event.preventDefault(); //prevent browser to reload page
    alert(inputs.username + inputs.age); //show input value in alert box
  };

  return (
    <div>
      <form onSubmit={handleSubmit}> // in onSubmit event call handelSubmit function
        <label> Enter your name:
        <input
          type="text"
          name="username"
          value={inputs.username || ""} //set input value from useState Object
          onChange={handleChange} //in onChange event call handleChange function to update
        /> </label>
        <label> Enter your age:
        <input
          type="number"
          name="age"
          value={inputs.age || ""} //set input value from useState Object
          onChange={handleChange} //in onChange event call handleChange function to update
        /> </label>
        <input type="submit" />
      </form>
    </div>
  );
}
```

3 App.jsx

```
import React from "react";
import Form from "../component/Form"; //import Form Component

function App() {
  return <Form />; //render the form component
}

export default App;
```

localhost:5173

Enter your name:

Enter your age:

localhost:5173 says
Anjesh Kumar20



1 Create folder name : **component** & in this folder create a file name : **Form.jsx**

2 component\Form.jsx

```
import { useState } from "react"; //import useState hook

export default function Form() {
  const [name, setName] = useState(""); //initialize usestate hook

  function handelSubmit(event) { //make function tiger when form submit
    event.preventDefault(); //prevent browser to reload page
    alert(`then name you enter was ${name}`); //show input value in alert box
  }

  return (
    <div>
      <form onSubmit={handelSubmit}> //in onSubmit event call handelSubmit function
        <label>First Name
          <input
            type="text"
            value={name}
            onChange={(e) => setName(e.target.value)} //using setName to update name variable
          />
        </label>
        <input type="submit" /> //submit button to submit form
      </form>
    </div>
  );
}
```

3 App.jsx

```
import React from "react";
import Form from "../component/Form"; //import Form Component

function App() {
  return <Form />; //render the form component
}

export default App;
```

- **Controlled Components:** Input values are tied to React state and updated via onChange.
- **Form Submission:** Handle form submission with onSubmit and prevent default behavior using event.preventDefault().
- **Handling Multiple Inputs:** Use a single state object to manage multiple form fields dynamically.
- **Validation:** Validate input either during onChange or onSubmit.

localhost:5173

First Name

localhost:5173 says
then name you enter was Anjesh Kumar

OK

From textarea

1 Create folder name : **component** & in this folder create a file name : **Form.jsx**

2 component\Form.jsx

```
import { useState } from "react"; //import useState hook

export default function Form() {
  const [textarea, setTextarea] = useState("Hello React Learner"); //initialize useState hook

  function handleSubmit(event) { //make function to trigger when form submit
    event.preventDefault(); //prevent browser to reload page
    alert(textarea); //show textarea data in alert box
  }

  return (
    <form onSubmit={handleSubmit}> //in onSubmit event call handleSubmit function

      <textarea
        value={textarea}
        onChange={(e) => { setTextarea(e.target.value);}}/> //onChange update textarea value
      <input type="submit" /> //submit button to submit form

    </form>
  );
}
```

3 App.jsx

```
import React from "react";
import Form from "../component/Form"; //import Form Component

function App() {
  return <Form />; //render the form component
}

export default App;
```

- In React, a textarea can be controlled using the useState hook to manage its value. The value prop of the textarea is linked to a state variable, and the onChange event updates the state when the user types in the textarea. This ensures the textarea content is dynamically managed by React. Here's a summary of the process:
- **useState:** Used to manage the textarea value.
- **value:** Binds the textarea to the state.
- **onChange:** Updates the state when the textarea content changes

localhost:5173

Hello React Learner

Submit

localhost:5173 says

Hello React learner

OK

1 Create folder name : **component** & in this folder create a file name : **Form.jsx**

2 component\Form.jsx

```
import { useState } from "react"; //import useState hook

export default function Form() {
  const [city, setCity] = useState("Delhi"); //initialize useSate hook

  function handleClick(event) { //make function tiger when form submit
    event.preventDefault(); //prevent browser to reload page
    alert(city);
  }

  return (
    <form> //using setCity to update city variable value
      <select value={city} onChange={(event) => setCity(event.target.value)}>
        <option value="Delhi">Delhi</option>
        <option value="Mumbai">Mumbai</option>
        <option value="Kolkata">Kolkata</option>
      </select>
      <button onClick={handleClick}>Click</button> //onClick event call handleClick function
    </form>
  );
}
```

3 App.jsx

```
import React from "react";
import Form from "../component/Form"; //import Form Component

function App() {
  return <Form />; //render the form component
}

export default App;
```

- In React, a textarea can be controlled using the useState hook to manage its value. The value prop of the textarea is linked to a state variable, and the onChange event updates the state when the user types in the textarea. This ensures the textarea content is dynamically managed by React. Here's a summary of the process:
- **useState:** Used to manage the textarea value.
- **value:** Binds the textarea to the state.
- **onChange:** Updates the state when the textarea content changes

localhost:5173

Delhi

Delhi

Mumbai

Kolkata

Click

localhost:5173 says

Delhi

OK

1 Create folder name : **component** & in this folder create a file name : **Header.jsx**

2 component\Header.jsx

```
function Header() {  
  return <h1 style={{ color: "red" }}>Hello React Learner</h1>;  
}  
  
// Note: In JSX, JavaScript expressions are written inside curly braces, and since JavaScript  
objects also use curly braces, the styling in the example above is written inside two sets of  
curly braces {{}}.  
-----  
  
function Header() {  
  return <h1 style={{ backgroundColor: "red" }}>Hello React Learner</h1>;  
}  
  
// Since the inline CSS is written in a JavaScript object, properties with hyphen separators,  
like background-color, must be written with camel case syntax:  
-----  
  
function Header() {  
  const myStyle = { color: "white", backgroundColor: "Blue", padding: "10px", };  
  
  return <h1 style={myStyle}>Hello React Learner</h1>;  
}  
  
// You can also create an object with styling information, and refer to it in the style  
attribute:  
  
export default Header
```

3 App.jsx

```
import React from "react";  
import Header from "../component/Header"; //import Form Component  
  
function App() {  
  return <Header />; //render the Header component  
}  
  
export default App;
```

localhost:5173

Hello React Learner

Hello React Learner

Hello React Learner

1 Create folder name : **style** inside assets folder & in this folder create a file name : **component.css**

2 Create folder name : **component** & in this folder create a file name : **Header.jsx**

3 component\Header.jsx

```
function Header() {  
  return <h1 className="Heading">Hello React Learner</h1>; //use className instead of class  
}  
export default Header;
```

4 assets\style\Component.css

```
.heading {  
  color: white;  
  background-color: blue; //css code  
  padding: 10px;  
}
```

5 App.jsx

```
import React from "react";  
import Header from "../component/Header"; //import header component  
import "../assets/style/Component.css"; //import style sheet  
  
function App() {  
  return <Header />; //render the Header component  
}  
  
export default App;
```

You can write your CSS styling in a separate file, just save the file with the .css file extension, and import it in your application.

Note: You can call the file whatever you like, just remember the correct file extension

localhost:5173

Hello React Learner

1 Create folder name : **style** inside assets folder & in this folder create a file name : **component.module.css**

2 Create folder name : **component** & in this folder create a file name : **Header.jsx**

3 component\Header.jsx

```
import styles from "../assets/style/Component.module.css"; //import css module file

function Header() {
  return <h1 className={styles.heading}>Hello React Learner</h1>;
} //in className attribute add heading style from styles object we import

export default Header;
```

4 assets\style\Component.module.css

```
.heading {
  color: white;
  background-color: blue; //css code
  padding: 10px;
}
```

5 App.jsx

```
import React from "react";
import Header from "../component/Header"; //import component

function App() {
  return <Header />; //render the component
}

export default App;
```

Another way of adding styles to your application is to use CSS Modules.
CSS Modules are convenient for components that are placed in separate files.

The CSS inside a module is available only for the component that imported it, and you do not have to worry about name conflicts.

Create the CSS module with the .module.css extension, example: **my-style.module.css**

localhost:5173

Hello React Learner

1 Create folder name : **component** & in this folder create a file name : **Hook.jsx**

2 component\Hook.jsx

```
import { useEffect, useState } from "react"; //import useEffect and useState hook

export default function Hook() {
  const [counter, setCounter] = useState(0); //initialize useState to hold and set counter
  const [name, setName] = useState("Anjesh"); //initialize useState to hold and set name

  useEffect(() => { //define useEffect,
    console.log(name); //when component render then inside useEffect statement execute
  }, []); //empty dependency array it's mean useEffect Statement execute only once

  useEffect(() => { //syntax of useEffect : useEffect(function(),[dependency])
    alert(counter);
  }, [counter]); //2nd example with dependency when counter value change re-execute useEffect

  return (
    <>
      <h1> Counter value is : {counter}</h1> //show counter value
      <button onClick={() => setCounter((counter) => counter + 1)}>Increase</button>
    </> //in onclick event increment counter value using setCounter
  );
}
```

3 App.jsx

```
import React from "react";
import Hook from "../component/Hook"; //include hook component

function App() {
  return <Hook />; //render the hook component
}

export default App;
```

Purpose: Manages side effects (e.g., data fetching, DOM manipulation, subscriptions) in function components. Runs After

Render: By default, useEffect runs after every render (initial render and updates).

Dependencies: The second argument is an array of dependencies. The effect runs only when those dependencies change. An empty array [] makes it run only once after the initial render.

Cleanup: If the effect returns a function, that function runs as cleanup before the component unmounts or the effect re-runs.

Common Use Cases: Fetching data, updating the DOM, setting timers, managing subscriptions (e.g., event listeners).

localhost:5173

Counter value is : 1

Increase

localhost:5173 says
1

OK

1 Create folder name : **component** & in this folder create 2 file name : **Hook.jsx** , **Message.jsx**

2 component\Hook.jsx

```
import { createContext, useState } from "react"; //import createContext, useState
import Message from "../Message";

const UserContext = createContext(); //create createContext, name must start with uppercase

export default function Hook() {
  const [user, setUser] = useState("Anjesh Kumar"); //initialize useState to hold & set user

  return (
    <>
      <UserContext.Provider value={user}> //use context provide to provide user to component
        <Message /> //render all component the need user data in side
      </UserContext.Provider> context provide in this example UserContext
    </>
  );
}
export { UserContext }; //export context in this example UserContext
```

3 component\Message.jsx

```
import { useContext } from "react"; //import useContext from react
import { UserContext } from "../Hook"; //import context in this example UserContext from Hook

export default function Message() {
  const user = useContext(UserContext); //initialize useContext, example useContext(context)

  return <h1>Hello user : {user}</h1>; //show user
}
```

4 App.jsx

```
import React from "react";
import Hook from "../component/Hook"; //import hook component

function App() {
  return <Hook />; //render hook component
}

export default App;
```

Purpose: Provides a way to pass data (or state) through the component tree without manually passing props at every level.

Context Provider: Data is made available to components via a Provider, which wraps the components that need access to the shared state or data.

useContext Hook: Allows components to directly access context values without using Consumer components, improving readability.

Shared State: Useful for global state management, such as themes, user authentication, avoiding "prop drilling."

Reactivity: When context value changes, components using that context will automatically re-render to reflect the updated value.

localhost:5173

Hello user : Anjesh Kumar

Hook useRef DOM Manipulation

20

1 Create folder name : **component** & in this folder create a file name : **Hook.jsx**

2 component\Hook.jsx

```
import { useRef } from "react"; //import useRef from react

export default function Hook() {
  const NameInput = useRef(); //initialize useRef, return object all data in current key

  function handleClick() { //define function tiger when button click
    NameInput.current.focus(); //using useRef to target DOM Element and manipulate
  }

  return (
    <>
      <input type="text" ref={NameInput} /> //using ref attribute to refer useRef
      <button onClick={handleClick}>focus</button> //in call handleClick function
    </>
  );
}
```

3 App.jsx

```
import React from "react";
import Hook from "../component/Hook"; //import hook component

function App() {
  return <Hook />; //render hook component
}

export default App;
```

Persistent Reference: The value stored in useRef persists across re-renders, unlike regular variables which reset on each render.

No Re-renders: Updating a ref does not trigger a re-render of the component.

Accessing DOM Elements: It's commonly used to directly access DOM elements in function components.

General Purpose Storage: You can use useRef to store any mutable value, not just DOM elements.

localhost:5173

focus

Hook useRef none UI impacting data

21

1 Create folder name : **component** & in this folder create a file name : **Hook.jsx**

2 component\Hook.jsx

```
import { useRef, useState } from "react"; //import useRef & useState

export default function Hook() {
  const countRef = useRef(0); //initialize useRef to hold counter value (countRef)
  const [count, setCount] = useState(0); //initialize useState to hold count value

  function handleClick() { //define function execute when button click
    countRef.current = countRef.current + 1; //add 1 in countRef : (countRef.current)
    console.log(countRef.current); //log the countRef value
    setCount((count) => count + 1); //using setCount() update count value,
  }

  return (
    <> //when useState data update useState will be re-render the component we can see
      <h1>useRef count : {countRef.current}</h1> countRef value
      <button onClick={handleClick}>Increment</button> //on click call handleClick function
    </>
  );
}
```

3 App.jsx

```
import React from "react";
import Hook from "../component/Hook"; //import hook component from component folder

function App() {
  return <Hook />; //render Hook component
}

export default App;
```

Definition: A React hook that stores a mutable value without causing re-renders.

Key Features:

No Re-renders: Changes to .current do not trigger a re-render.

Persistence: Retains value across renders.

Cases:

Track values (timers, counters), Access DOM elements,

Improve performance by avoiding unnecessary updates.

Comparison:

Unlike useState, it does not cause re-renders, making it ideal for non-UI impacting data.

localhost:5173

useRef count : 0

Increment

1 Create folder name : **component** & in this folder create a file name : **Counter.jsx**

2 component\Counter.jsx

```
import { useReducer } from "react";

// Initial state
const initialState = 0;

// Reducer function
const reducer = (state, action) => {
  switch (action.type) {
    case "INCREMENT":
      return state + 1;
    case "DECREMENT":
      return state - 1;
    default:
      return state; // If action is unknown, return current state
  }
};

function Counter() {
  // useReducer returns the current state and a dispatch function
  const [count, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch({ type: "INCREMENT" })}>Increment</button>
      <button onClick={() => dispatch({ type: "DECREMENT" })}>Decrement</button>
    </div>
  );
}

export default Counter;
```

3 App.jsx

```
import React from "react";
import Counter from "../component/Counter"; //import Counter Component

function App() {
  return <Counter />; //render counter component
}

export default App;
```

localhost:5173

Count: 0

Increment

Decrement

1 Create folder name : **component** & in this folder create a file name : **Calculator.jsx**

2 component\Calculator.jsx

```
import React, { useState, useMemo } from "react";

function Calculator() {
  const [count, setCount] = useState(0);
  const [otherCount, setOtherCount] = useState(0);

  // A simple expensive function
  function calculateDouble(num) {
    console.log("Calculating...");
    for (let i = 0; i < 1000000000; i++) {} // Simulate a heavy task
    return num * 2;
  }

  // useMemo to cache the result : useMemo(function(),[dependency])
  const doubledValue = useMemo(() => calculateDouble(count), [count]);

  return (
    <div>
      <p>Count: {count}</p> <p>Doubled Value: {doubledValue}</p>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>
      <p>Other Count: {otherCount}</p>
      <button onClick={() => setOtherCount(otherCount + 1)}>Increment Other Count </button>
    </div>
  );
}

export default Calculator;
```

3 App.jsx

```
import React from "react";
import Calculator from "../component/Calculator"; //import Calculator component

function App() {
  return <Calculator />; //render component
}

export default App;
```

In short, useMemo is useful for optimizing components that perform expensive calculations, ensuring that they only recalculate when necessary.

With useMemo: The result of the calculation is "memoized" (i.e., stored), so it only runs again if the specified dependencies change. This prevents unnecessary recalculations during re-renders, making the component more efficient.

localhost:5173

Count: 0
Doubled Value: 0

Increment Count

Other Count: 0

Increment Other Count

1 Create folder name : **component** & in this folder create a file name : **Counter.jsx**

2 component\Counter.jsx

```
import React, { useState, useCallback } from "react"; //import useState & useCallback

function Counter() {
  const [count, setCount] = useState(0); //initialize useState to hold & set count value
  const [text, setText] = useState(""); //initialize useState to hold & set text value

  const increment = useCallback(() => { //define a useCallback(function(), [dependency]) hook
    console.log("Increment function created");
    setCount((prevCount) => prevCount + 1); //update count value
    console.log("count update");
  }, [count]);

  const handleChange = useCallback((e) => { //define a useCallback(function(), [dependency]) hook
    console.log("handleChange function created");
    setText(e.target.value); //update text value
    console.log("input change");
  }, []);

  return (
    <div>
      <input type="text" value={text} onChange={handleChange} />
      <button onClick={increment}>Increment</button>
      <p>Count: {count}</p>
    </div>
  );
}
export default Counter;
```

3 App.jsx

```
import React from "react";
import Counter from "../component/Counter";

function App() {
  return <Counter />;
}

export default App;
```

The useCallback hook in React memoizes a function to prevent it from being recreated on every render. It only changes when its dependencies change, improving performance by avoiding unnecessary re-renders in child components that depend on the function.

Parameters:

Callback function: This is the function you want to memoize. Dependencies array: A list of values that, when changed, will cause the callback function to be re-created.

localhost:5173

Increment

Count: 0

Hook Custom hook : useFetchData

1 Create folder name : **component** & in this folder create a file name : **useFetchData.jsx** & **MyComponent**

2 component\useFetchData.js

```
import { useState, useEffect } from "react";

function useFetchData(url) { // This is the custom hook
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(url) // Fetch data when the hook is used
    .then((response) => response.json())
    .then((data) => {
      setData(data);
      setLoading(false);
    });
  }, [url]);

  return { data, loading }; // Return the fetched data and loading state
}

export default useFetchData;
```

3 component\useFetchData.js

```
import React from "react";
import useFetchData from "../useFetchData";

function MyComponent() {
  // Using the custom hook to fetch data
  const { data, loading } = useFetchData("https://jsonplaceholder.typicode.com/users");

  if (loading) {
    return <div>Loading...</div>;
  }

  return <div> <pre>{JSON.stringify(data, null, 2)}</pre> </div>
}

export default MyComponent;
```

4 App.jsx

```
import React from "react";
import MyComponent from "../component/MyComponent"; //import myComponent

function App() {
  return <MyComponent />; //render my component
}

export default App;
```

Here's a simple breakdown:

Regular Hook: React provides built-in hooks like `useState` and `useEffect` to add functionality to components.

Custom Hook: When you notice that you are using the same logic in different places (e.g., fetching data from an API), you can move that logic to a custom hook. This custom hook is just a function that uses other hooks inside it.

Steps to create a custom hook:

- 1) Create a function starting with the word `use` (e.g., `useFetchData`).
- 2) Inside the function, use any built-in hooks you need, like `useState` or `useEffect`.
- 3) Return the values or functions you want to share from the custom hook.